

Jobify Database Deep Explanation Guide

This document explains the complete database architecture of the Jobify project. It is designed for viva, interviews, and evaluation preparation.

1. Database Overview

The Jobify database is centered around the users table. Every major feature such as jobs, community posts, resume building, and payments is connected directly or indirectly to a user.

```
users
├── jobs
│   └── community_posts
│       └── resumes
│           └── payments
```

2. Users Table

```
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  email VARCHAR(100) UNIQUE,
  password VARCHAR(255),
  subscription_type ENUM('free','premium') DEFAULT 'free',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

The users table stores authentication and subscription information. The email field is unique to prevent duplicate accounts. Password uses VARCHAR(255) because hashed passwords are longer.

3. Jobs Table

```
CREATE TABLE jobs (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT,
  company VARCHAR(100),
  role VARCHAR(100),
  work_type ENUM('remote','onsite','hybrid'),
  salary_range VARCHAR(50),
  status ENUM('saved','applied','interviewing','offer','rejected') DEFAULT 'saved',
  date_applied DATE,
  location VARCHAR(100),
  job_posting_url VARCHAR(255),
  notes TEXT,
  interview_questions TEXT,
  experience TEXT,
  share_in_community TINYINT DEFAULT 0,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);
```

The jobs table is the core functionality of the project. Each row represents one tracked job application. ON DELETE CASCADE ensures jobs are removed if a user is deleted.

4. Community Posts Table

```
CREATE TABLE community_posts (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  user_id INT,  
  job_id INT,  
  company VARCHAR(100),  
  role VARCHAR(100),  
  question TEXT,  
  experience TEXT,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,  
  FOREIGN KEY (job_id) REFERENCES jobs(id) ON DELETE CASCADE  
);
```

Community posts store interview experiences shared publicly. This is a derived table because data originates from jobs.

5. Resume Table

```
CREATE TABLE resumes (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  user_id INT UNIQUE,  
  full_name VARCHAR(100),  
  email VARCHAR(100),  
  phone VARCHAR(20),  
  linkedin VARCHAR(255),  
  skills TEXT,  
  education JSON,  
  projects JSON,  
  experience JSON,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE  
);
```

Resume data uses JSON fields because education, projects, and experience sections are dynamic and flexible.

6. Payments Table

```
CREATE TABLE payments (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  user_id INT,  
  razorpay_order_id VARCHAR(100),  
  razorpay_payment_id VARCHAR(100),  
  status ENUM('created', 'paid', 'failed') DEFAULT 'created',  
  amount INT,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE  
);
```

The payments table tracks payment lifecycle from creation to successful verification and premium upgrade.

7. Complete Flow

Register → users

Login → JWT verification

Add Job → jobs
→ optional community_posts

Resume → resumes

Upgrade → payments
→ users.subscription_type updated

8. Viva Questions

Q: Why use JSON in resumes?

A: Because resume sections are flexible and variable.

Q: Why separate community_posts?

A: To isolate public data from private jobs.

Q: Why use ON DELETE CASCADE?

A: To prevent orphan records.

Q: What happens when user shares a job?

A: Backend inserts data into community_posts.

9. Conclusion

The Jobify database follows proper relational design principles with foreign keys, structured flows, and scalable architecture.